Practical 3 - Implementation of Classification algorithm in R Programming.

- **What are Classification Models?**
- Classification models are supervised learning methods designed to predict the value of a **categorical target attribute**, unlike regression models which focus on numerical outcomes.
- By learning from past observations with known class labels, classification models generate rules that can be applied to predict the class of future examples.
- Classification occupies a central role in learning theory, both for its theoretical significance and its wide-ranging practical applications.
- From a theoretical perspective, the ability of algorithms to learn from experience represents a crucial step toward emulating the inductive reasoning capabilities of the human brain.

- **What Is a Classification Problem?**
- A **classification problem** involves predicting a **categorical target attribute** (also called a class or label) based on a set of **explanatory attributes** (also called features or predictors), which can be numerical or categorical.
- **What is Decision Tree?**
- **A decision tree is a supervised learning algorithm that uses a tree-like model of decisions and their possible outcomes, commonly applied in classification and regression tasks.** It visually represents choices, conditions, and results, making it intuitive for both technical and non-technical audiences

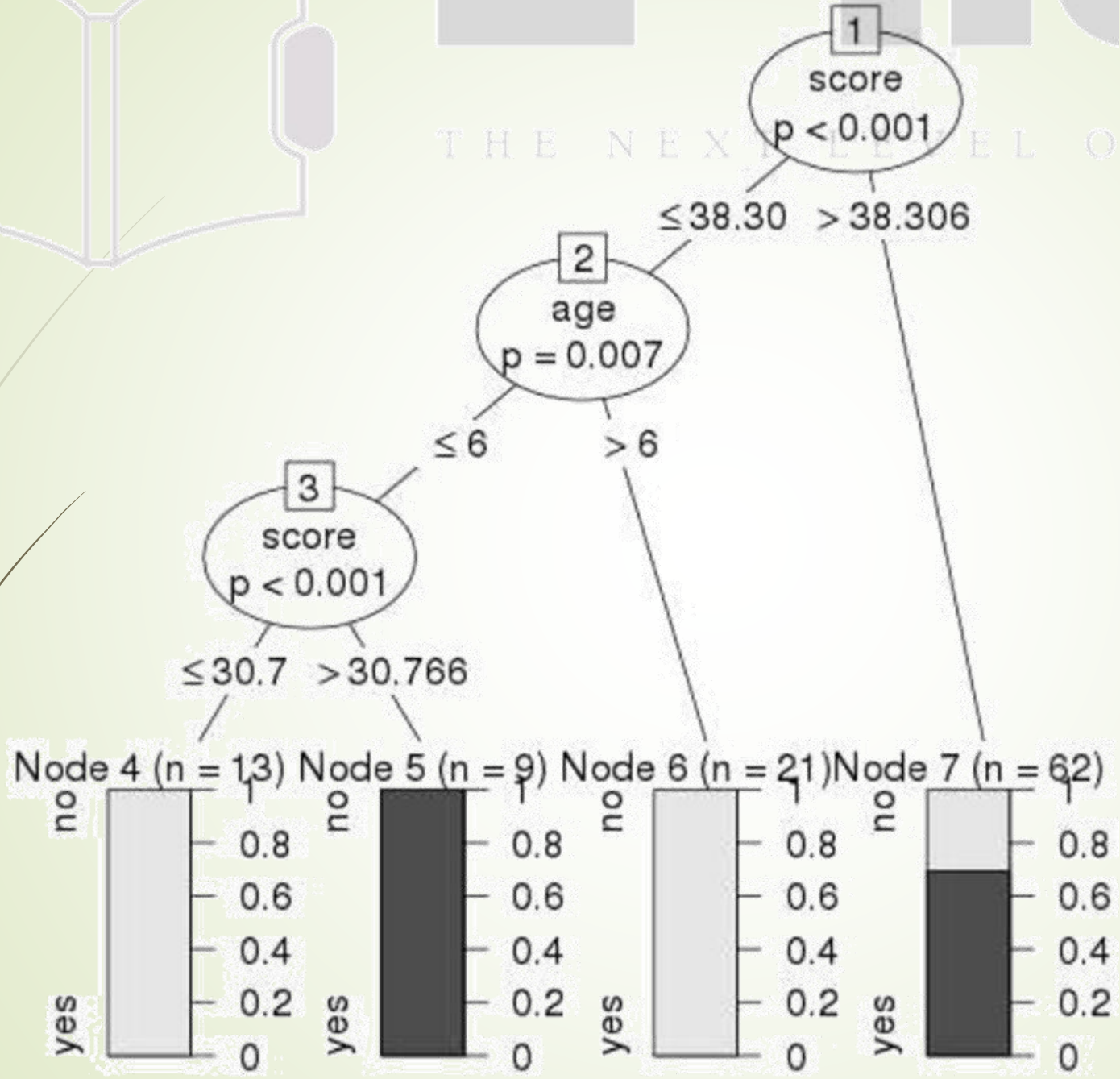
- 1. Consider the annual rainfall details at a place starting from January 2012. We create an R time series object for a period of 12 months and plot it.

```
# Get the data points in form of a R vector.
rainfall <-
c(799,1174.8,865.1,1334.6,635.4,918.5,685.5,998.6,784.2,985,882.8,1071)
# Convert it to a time series object.
rainfall.timeseries <- ts(rainfall,start = c(2012,1),frequency = 12)
# Print the timeseries data.
print(rainfall.timeseries)
# Give the chart file a name.
png(file = "rainfall.png")
# Plot a graph of the time series.
plot(rainfall.timeseries)
# Save the file.
dev.off()
```

- 2. Practical Implementation of Decision Tree using R Tool
- We will use the R in-built data set named readingSkills to create a decision tree. It describes the score of someone's readingSkills if we know the variables "age","shoesize","score" and whether the person is a native speaker or not.

- install.packages("party")

- Load the party package. It will automatically load other
- # dependent packages.
- library(party)
- # Create the input data frame.
- input.dat <- readingSkills[c(1:105),]
- # Give the chart file a name.
- png(file = "decision_tree.png")
- # Create the tree.
- output.tree <- ctree(
- nativeSpeaker ~ age + shoeSize + score,
- data = input.dat)
- # Plot the tree.
- plot(output.tree)
- # Save the file.
- dev.off()


1
score
p < 0.001
≤ 38.30
> 38.306
2
age
p = 0.007
≤ 6
> 6
3
score
p < 0.001
≤ 30.7
> 30.766
Node 4 (n = 13)
Node 5 (n = 9)
Node 6 (n = 21)
Node 7 (n = 62)
no
yes
1
0.8
0.6
0.4
0.2
0

Practical 4 - Perform the data clustering using clustering algorithm using Python.

What is Clustering?

- **Unsupervised Learning**: Unlike classification, clustering doesn't rely on predefined labels.
- **Similarity-Based Grouping**: Data points are grouped using similarity or distance measures (e.g., Euclidean distance, cosine similarity).
- **Purpose**: Helps discover natural groupings, hidden patterns, or structures in complex datasets.
- **Applications**: Customer segmentation, anomaly detection, document organization, image recognition, and biological data analysis.

Clustering Algorithms

Here are the most widely used clustering methods

Algorithm	Approach	Strengths	Limitations
K-Means	Partitions data into k clusters by minimizing variance within clusters.	Simple, fast, works well with spherical clusters.	Sensitive to outliers, requires predefining k.
Hierarchical Clustering	Builds a tree (dendrogram) of clusters by merging or splitting.	No need to predefine k, good for small datasets.	Computationally expensive for large datasets.
DBSCAN (Density-Based Spatial Clustering of Applications with Noise)	Groups points in dense regions, marking outliers as noise.	Finds arbitrarily shaped clusters, handles noise well.	Struggles with varying densities, parameter tuning needed.
Mean-Shift	Iteratively shifts data points toward dense regions.	Does not require k, good for arbitrary shapes.	Computationally heavy, sensitive to bandwidth parameter.
Gaussian Mixture Models (GMM)	Assumes data is generated from a mixture of Gaussian distributions.	Flexible, probabilistic clustering.	Assumes Gaussian distribution, can be complex to train.
Agglomerative Clustering	Bottom-up hierarchical method merging closest clusters.	Intuitive, produces dendrograms.	Slow for large datasets.
Spectral Clustering	Uses graph theory and eigenvalues of similarity matrix.	Effective for non-convex clusters.	Computationally expensive, requires similarity matrix.

1) Write a Python program for k-means clustering.

```
# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Step 2: Generate synthetic dataset
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# Step 3: Apply K-Means clustering
kmeans = KMeans(n_clusters=4, random_state=0)
kmeans.fit(X)

# Step 4: Get cluster labels and centroids
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
```

```
# Step 5: Visualize clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=200, alpha=0.7, marker='X')
plt.title("K-Means Clustering Example")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

2) Write a python code for DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

```
# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN
```

```
# Step 2: Generate synthetic dataset
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.50, random_state=0)

# Step 3: Apply DBSCAN clustering
dbscan = DBSCAN(eps=0.5, min_samples=5)
# eps = neighborhood radius, min_samples = minimum points in cluster
labels = dbscan.fit_predict(X)

# Step 4: Visualize clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='plasma', s=50)
plt.title("DBSCAN Clustering Example")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```